

CSCE 605 COMPILER DESIGN · MODULE 10 · LECTURE NOTES

Code Generation

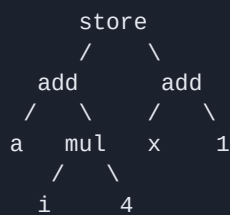
From IR to instructions the machine actually has.

MODULE OUTCOMES

- Explain instruction selection as tree tiling.
- Apply dynamic programming to select optimally over a tree.
- Explain list scheduling and what it is scheduling for.
- Explain calling conventions and why they are a contract.
- Explain what debug information must survive optimisation.

1 Instruction selection

IR for `a[i] = x + 1`



TILES on x86-64:

<code>add r, r</code>	cost 1	
<code>lea r, [r + r*4]</code>	cost 1	<- covers THREE IR nodes
<code>mov [r + r*4], r</code>	cost 1	<- covers FOUR
<code>imul r, imm</code>	cost 3	

Naive one-node-per-instruction: 5 instructions.

Good covering using addressing modes: 2 instructions.

Selection = COVER THE TREE WITH TILES AT MINIMUM COST.

OPTIMAL ON TREES, INTRACTABLE ON GRAPHS

Over a tree, dynamic programming produces the provably optimal cover in linear time. Compute, bottom-up, the minimum cost to produce each node's value in each possible storage class; then walk back down selecting the tiles that achieved it. This is a clean and satisfying result.

Over a DAG the problem is NP-hard, because a shared subtree may want to be covered by different tiles on behalf of its different parents, and the choices interact.

And real IR is a DAG. Common subexpressions are shared by construction, and CSE and GVN (Module 09) deliberately create more sharing — so the easy case is exactly the one that does not occur.

So compilers split the DAG into trees at the shared nodes and accept suboptimality, or use heuristic matchers, or generate a matcher from a machine description. **LLVM's SelectionDAG does roughly the first; GlobalISel is a different and ongoing bet** on a more incremental approach. That a production compiler is still replacing its instruction selector after twenty years is a reasonable indication of how unsettled this area is.

2 Scheduling

SCHEDULING HIDES LATENCY THE HARDWARE CANNOT

Instructions have latencies. A load from L1 takes several cycles, from memory hundreds (CSCE 614); a divide takes tens. An instruction that consumes a result must wait for it.

So reorder to place independent work between a producer and its consumer. List scheduling does this greedily: build the dependence DAG, repeatedly pick a ready instruction by some priority, and emit it.

The priority is usually critical path length — schedule first whatever has the most dependent work behind it, since delaying it delays everything.

Out-of-order hardware already does this at runtime (CSCE 614), which makes compile-time scheduling far less critical on large desktop and server cores — and **it remains essential on in-order cores, on VLIW machines where the compiler must fill the issue slots explicitly, and on GPUs** (CSCE 735 Module 08). **That is most of the processors in the world by count**, so scheduling has not become irrelevant; it has become target-dependent.

SCHEDULING AND REGISTER ALLOCATION HAVE OPPOSED GOALS

Scheduling wants to spread dependent computations apart so that independent work fills the latency — which necessarily lengthens the live ranges of the values involved.

Register allocation wants live ranges short, so that fewer values are simultaneously live and the allocation fits the register file without spilling (Module 11).

So scheduling before allocation produces spills that the scheduler's improvement does not pay for, while **allocating before scheduling constrains the scheduler** with false dependences through reused physical registers that were not in the original program at all.

The usual answer is to schedule, allocate, then schedule again — a pre-pass scheduler, the allocator, and a post-pass scheduler to clean up. **It is a hack, it is standard practice in every production compiler, and it is Module 09's phase-ordering problem in its most concrete and least deniable form:** two phases whose optimal choices genuinely depend on each other, resolved by running one of them twice.

3 Calling conventions

Decision	System V AMD64 (Linux, macOS, BSD)
Integer argument registers, in order	rdi, rsi, rdx, rcx, r8, r9 , then the stack right to left.
Floating-point arguments	xmm0 through xmm7.
Return value	rax; rdx:rax for 128-bit; xmm0 for floating point.
Caller-saved (volatile)	rax, rcx, rdx, rsi, rdi, r8–r11 . The caller must save these if it needs them across a call.
Callee-saved (non-volatile)	rbx, rbp, r12–r15 . The callee must restore them before returning — which interacts directly with register allocation (Module 11).
Stack alignment	16 bytes at the point of the call , which means the callee sees the return address making it 8 mod 16. Getting this wrong produces crashes only in functions using aligned SIMD.
Struct passing	By classifying each eightbyte of the struct as INTEGER, SSE, or MEMORY, with aggregates over 16 bytes passed in memory. The subtle part, and where real ABI bugs live.
Variadic functions	al must hold the number of vector registers used — an obscure rule that breaks printf when violated.

THE ABI IS WHY YOU CANNOT JUST CHANGE THINGS

Separately compiled code must agree on all of it. Your compiler, the system C library, the operating system's syscall stubs, and every third-party binary were built at different times by different people and must interoperate exactly.

So a better calling convention cannot be adopted unilaterally. Everything would have to change together, which in practice means nothing changes — the coordination cost exceeds any plausible benefit.

Which is why ABIs long outlive the hardware they were designed for. The System V AMD64 register assignment was fixed around 2000 and is still what every Linux binary uses, on processors with completely different microarchitectural characteristics.

And it is why C is the universal interoperability layer. Not because C is a good language for describing interfaces — it is not, having no modules, no ownership, and a weak type system — but because **the C ABI is the one that every language and every platform already implements.** Rust, Python, Go, and Swift all speak to each other in C, through a convention none of them would have chosen.

4 Debug information

OPTIMISATION DESTROYS THE CORRESPONDENCE TO SOURCE

A debugger needs two mappings: machine address to source line, and variable name to storage location. Both are straightforward in unoptimised code and both are destroyed by optimisation.

Instructions are reordered across source lines (§2), so stepping jumps backwards and forwards. **A variable lives in different registers at different points**, or in a register for part of its life and memory for another, or **is eliminated entirely**. **Inlined code comes from a different function and a different file**, so one address belongs to several source locations at once.

DWARF handles this with location lists — a variable's location expressed as a function of the program counter, rather than a single answer — plus explicit inlining records giving the inlined-at chain so a stack trace can be reconstructed.

'Optimised out' is the honest answer when a variable genuinely has no storage at that point, and a debugger saying so is behaving correctly. **Carrying debug information accurately through every optimisation pass is substantial ongoing engineering work**, and it is easy to get silently wrong.

- **Preserve the source location when moving an instruction**, and do not invent one for an instruction you created.
- **Drop a location rather than attach a plausible-looking wrong one**. A debugger that stops on the wrong line actively misleads; one that says it does not know is merely unhelpful.
- **Record inlining explicitly** — the inlined-at chain — so that a stack trace through inlined frames can be reconstructed. Without it, inlining makes profiles and crash reports unattributable.
- **Never let debug information change code generation**. Compiling with -g must produce byte-identical code to compiling without it. **If it does not, then debugging changes the bug**, which is the worst possible property for a debugging aid.
- **And test it. Debug information has no runtime behaviour, so nothing fails when it rots** — no test goes red, no benchmark regresses, and the quality degrades silently across releases until someone tries to debug an optimised build and finds every variable optimised out. This is why projects that care about it run dedicated debug-info test suites.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Appel — Modern Compiler Implementation, chapter 9 (instruction selection)	Tree tiling and the dynamic programming algorithm of §1 , presented clearly.
System V AMD64 ABI specification (free)	The §3 contract, normatively . The struct classification section is the one worth reading carefully.

Resource	What it covers for this module
Cooper & Torczon — Engineering a Compiler, chapters 11 and 12	Selection and scheduling, with the DAG complexity result.
DWARF Debugging Standard (free)	Location lists and inlining records — the §4 mechanism. Dense; read the introduction and the location-list chapter.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement tree tiling with dynamic programming for a small instruction set.
- 2 **Add x86 addressing-mode tiles** and measure the reduction in instruction count.
- 3 Construct a DAG where tree splitting produces a suboptimal cover, and quantify the loss.
- 4 Implement list scheduling with critical-path priority.
- 5 **Measure scheduling's effect on an in-order target** and on an out-of-order one, and explain the difference.
- 6 **Demonstrate the scheduling/allocation conflict:** schedule aggressively, then allocate, and count the spills.
- 7 Implement the System V calling convention and call into C from your generated code.
- 8 **Deliberately violate the stack alignment rule** and find a function that crashes because of it.
- 9 Emit DWARF line information and step through your generated code in gdb.
- 10 **Optimise a function and observe which variables become 'optimised out'.** Verify that -g did not change the code.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe instruction selection as tiling and say why addressing modes matter.
- 2 Why is tiling optimal on trees and NP-hard on DAGs, and why does that matter?
- 3 What is scheduling for, and on which targets does it matter most?
- 4 Why do scheduling and register allocation conflict, and how is it resolved?
- 5 Name six things a calling convention fixes.
- 6 Why can an ABI not be changed unilaterally, and what follows?
- 7 How does optimisation break debugging, and what does DWARF do about it?
- 8 Give five debug-info rules for a pass author.

